System and Network Security

Dr. Ashok Kumar Das

IEEE Senior Member
Professor
Center for Security, Theory and Algorithmic Research
International Institute of Information Technology, Hyderabad

E-mail: *ashok.das@iiit.ac.in*
Homepage: `http://www.iiit.ac.in/people/faculty/ashok-kumar-das`
Personal Homepage: `https://sites.google.com/view/iitkgpakdas/`

Authentication Applications: KERBEROS

Problem that Kerberos was designed to address

- Kerberos is an *authentication service* developed as part of Project Athena at MIT, USA.
- Suppose an open distributed environment in which users at workstations wish to access services on servers distributed throughout the network.
- We want for servers to be able to restrict access to authenticate requests for service.
- In this environment, a workstation cannot be trusted to identify its users correctly to network services.

Threats associated with user authentication over a network or Internet

- A user may gain access to a particular workstation (client) and pretend to be another user operating from that workstation. **(User impersonation attacks)**
- A user may alter the network address of a workstation so that the requests send from the altered workstation appear to come from the impersonated workstation. **(Forgery attacks)**
- A user may eavesdrop on exchanges of information and use a replay attack to gain entrance to a server or to disrupt operations.

Requirements for Kerberos

- **Secure:** A network eavesdropper should not be able to obtain the necessary information to impersonate a user.
- **Reliable:** Kerberos should be highly reliable and should employ a distributed server architecture, with one system able to back up another.
- **Transparent:** Ideally, the user should not be aware that authentication is taking place, beyond the requirement to enter a password.
- **Scalable:** The system should be capable of supporting a large number of clients and servers. This suggests a modular, distributed architecture.

Characteristics of Kerberos

- Allows a process (a client) running on behalf of a principal (a user) to prove its identity to a verifier (an application server, or just a server).
- Kerberos is not effective against password guessing attacks.
- Kerberos requires a trusted path through which passwords are entered.
- Kerberos is a distributed authentication service.
- It is developed as a part of Project Athena at MIT in the mid-1980.
- Kerberos v5 (version 5) is considered to be a standard Kerberos.
- Kerberos v4 is easy to explain.
- Use of timestamps provides a safeguard for Kerberos.
- Kerberos provides cross-realm authentication.
- Kerberos is based in part on the Needham-Schroder authentication protocol.

Overview of Kerberos

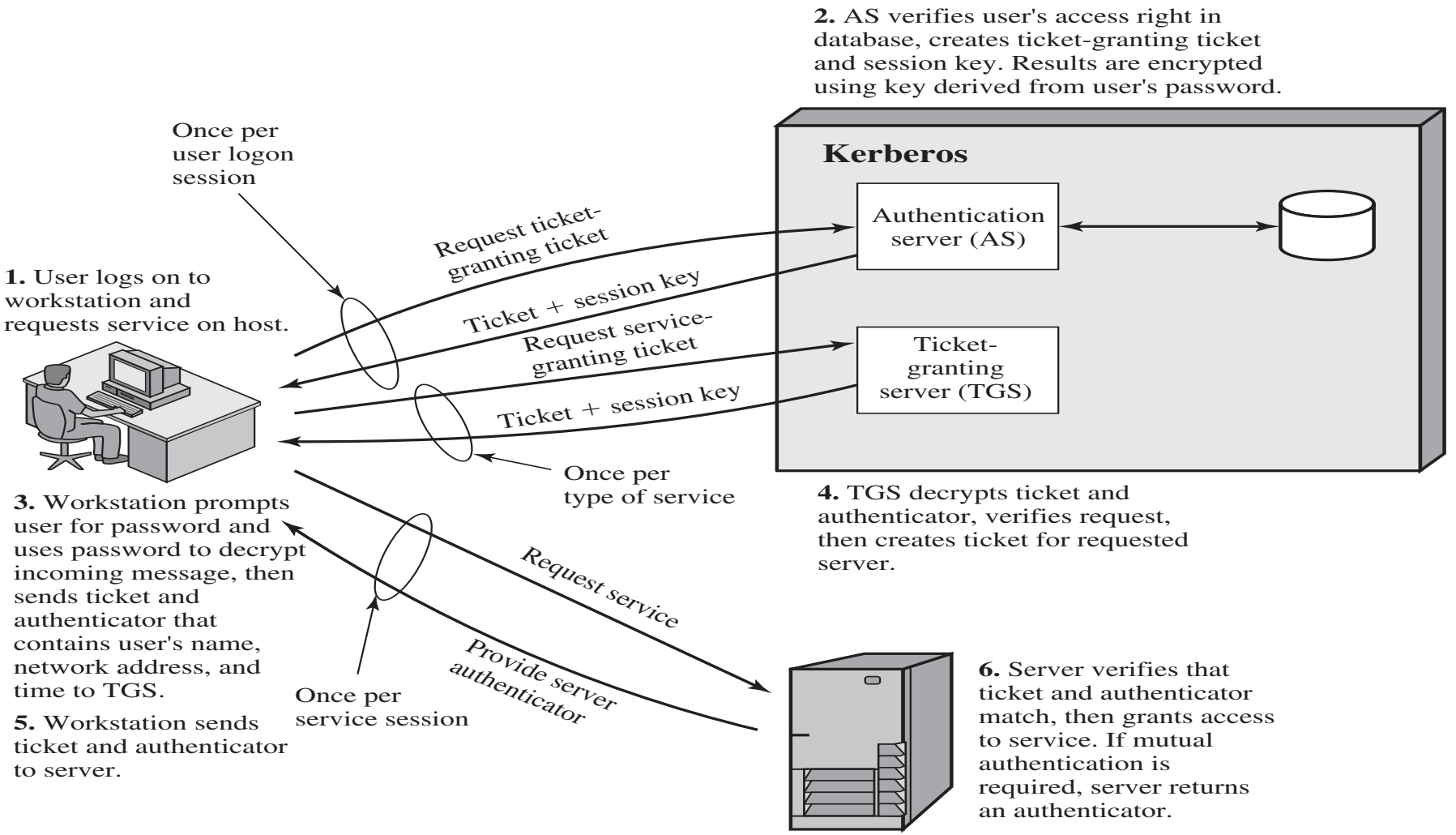

Figure: Overview of Kerberos v4

Overview of Kerberos (Continued...)

- (Once per user logon session)
  - 1. User logs on to workstation and requests service on host.
  - 2. AS verifies user's access right in database, creates ticket-granting ticket and session key. Results are encrypted using key derived from user's password.
- (Once per type of service)
  - 3. Workstation prompts user for password and uses password to decrypt incoming message, then send ticket and authentication that contains user's name, network address, and time to TGS.
  - 4. TGS decrypts ticket and authenticator, verifies request, then creates ticket for requested server.
- (Once per service session)
  - 5. Workstation sends ticket and authenticator to server.
  - 6. Server verifies that ticket and authenticator match, then grants access to service. If mutual authentication is required, server returns an authenticator.

Rationale for Elements of Kerberos Version 4 Protocol

Message (1)	Client requests ticket-granting ticket.
ID_C	Tells AS identity of user from this client.
ID_{tgs}	Tells AS that user requests access to TGS.
TS_1	Allows AS to verify that client's clock is synchronized with that of AS.
Message (2)	AS returns ticket-granting ticket.
K_c	Encryption is based on user's password, enabling AS and client to verify password, and protecting contents of message (2).
$K_{c,tgs}$	Copy of session key accessible to client created by AS to permit secure exchange between client and TGS without requiring them to share a permanent key.
ID_{tgs}	Confirms that this ticket is for the TGS.
TS_2	Informs client of time this ticket was issued.
$Lifetime_2$	Informs client of the lifetime of this ticket.
$Ticket_{tgs}$	Ticket to be used by client to access TGS.

(a) Authentication Service Exchange

Message (3)	Client requests service-granting ticket.
ID_V	Tells TGS that user requests access to server V.
$Ticket_{tgs}$	Assures TGS that this user has been authenticated by AS.
$Authenticator_c$	Generated by client to validate ticket.
Message (4)	TGS returns service-granting ticket.
$K_{c,tgs}$	Key shared only by C and TGS protects contents of message (4).
$K_{c,v}$	Copy of session key accessible to client created by TGS to permit secure exchange between client and server without requiring them to share a permanent key.
ID_V	Confirms that this ticket is for server V.
TS_4	Informs client of time this ticket was issued.
$Ticket_V$	Ticket to be used by client to access server V.
$Ticket_{tgs}$	Reusable so that user does not have to reenter password.
K_{tgs}	Ticket is encrypted with key known only to AS and TGS, to prevent tampering.
$K_{c,tgs}$	Copy of session key accessible to TGS used to decrypt authenticator, thereby authenticating ticket.
ID_C	Indicates the rightful owner of this ticket.
AD_C	Prevents use of ticket from workstation other than one that initially requested the ticket.
ID_{tgs}	Assures server that it has decrypted ticket properly.
TS_2	Informs TGS of time this ticket was issued.
$Lifetime_2$	Prevents replay after ticket has expired.
$Authenticator_c$	Assures TGS that the ticket presenter is the same as the client for whom the ticket was issued has very short lifetime to prevent replay.

Rationale for Elements of Kerberos Version 4 Protocol

$K_{c,tgs}$	Authenticator is encrypted with key known only to client and TGS, to prevent tampering.
ID_C	Must match ID in ticket to authenticate ticket.
AD_C	Must match address in ticket to authenticate ticket.
TS_3	Informs TGS of time this authenticator was generated.

(b) Ticket-Granting Service Exchange

Message (5)	Client requests service.
$Ticket_V$	Assures server that this user has been authenticated by AS.
$Authenticator_c$	Generated by client to validate ticket.
Message (6)	Optional authentication of server to client.
$K_{c,v}$	Assures C that this message is from V.
$TS_5 + 1$	Assures C that this is not a replay of an old reply.
$Ticket_v$	Reusable so that client does not need to request a new ticket from TGS for each access to the same server.
K_v	Ticket is encrypted with key known only to TGS and server, to prevent tampering.
$K_{c,v}$	Copy of session key accessible to client; used to decrypt authenticator, thereby authenticating ticket.
ID_C	Indicates the rightful owner of this ticket.
AD_C	Prevents use of ticket from workstation other than one that initially requested the ticket.
ID_V	Assures server that it has decrypted ticket properly.
TS_4	Informs server of time this ticket was issued.
$Lifetime_4$	Prevents replay after ticket has expired.
$Authenticator_c$	Assures server that the ticket presenter is the same as the client for whom the ticket was issued; has very short lifetime to prevent replay.
$K_{c,v}$	Authenticator is encrypted with key known only to client and server, to prevent tampering.
ID_C	Must match ID in ticket to authenticate ticket.
AD_C	Must match address in ticket to authenticate ticket.
TS_5	Informs server of time this authenticator was generated.

(c) Client/Server Authentication Exchange

Summary of Kerberos v4 Message Exchanges

- (a) Authentication Service Exchange: to obtain ticket-granting ticket
  - (1) $C \rightarrow AS : ID_c||ID_{tgs}||TS_1$, where TS is the timestamp.
  - (2) $AS \rightarrow C : E_{K_c}[K_{c,tgs}||ID_{tgs}||TS_2||lifetime_2||Ticket_{tgs}]$, where $Ticket_{tgs} = E_{K_{tgs}}[K_{c,tgs}||ID_c||AD_c||ID_{tgs}||TS_2||lifetime_2]$; AD_c is the network address of client C, K_c the encryption key based on user's password, $K_{c,tgs}$ the session key between C and TGS. Lifetime prevents replay attack after ticket has expired.
- (b) Ticket-granting Service Exchange: to obtain service-granting ticket
  - (3) $C \rightarrow TGS : ID_v||Ticket_{tgs}||Authenticator_c$, where $Authenticator_c = E_{K_{c,tgs}}[ID_c||AD_c||TS_3]$.
  - (4) $TGS \rightarrow C : E_{K_{c,tgs}}[K_{c,v}||ID_v||TS_4||Ticket_v]$, where $Ticket_v = E_{K_v}[K_{c,v}||ID_c||AD_c||ID_v||TS_4||lifetime_4]$.

Summary of Kerberos v4 Message Exchanges (Continued...)

- (c) Client/Server Authentication Exchange: to obtain service
  - (5) $C \rightarrow V : Ticket_v || Authenticator_c$, where $Authenticator_c = E_{K_{c,v}}[ID_c || AD_c || TS_5]$.
  - (6) $V \rightarrow C : E_{K_{c,v}}[TS_5 + 1]$, for mutual authentication, if required.

Cross-realm Authentication

- A full-service Kerberos environment consisting of a Kerberos server, a number of clients, and a number of application servers requires the following:
  1) The Kerberos server must have the user ID (UID) and hashed password of all participating users in its database. All users are registered with the Kerberos server.
  2) The Kerberos server must share a secret key with each server. All servers are registered with the Kerberos server.

  Such an environment is referred to as a realm.
  Example: Networks of clients and servers under different administrative organizations typically constitute different realms.

- ***Problem:*** **How can a client in one realm access service to a server located in another realm?**

Cross-realm Authentication (Continued...)

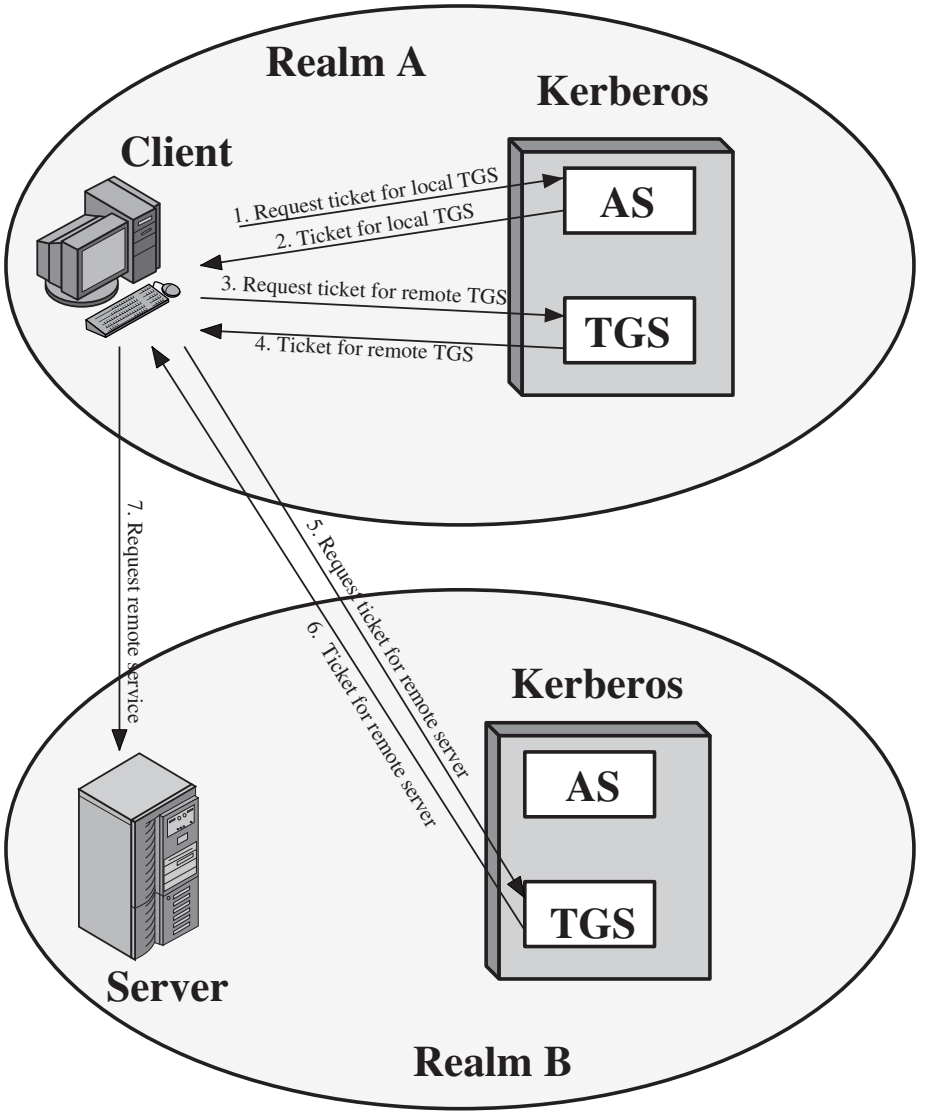

Figure: Request for service in another realm (Cross-realm Authentication)

Involved Steps in Cross-realm Authentication

- **Step 1.** Request ticket for local TGS
- **Step 2.** Ticket for local TGS
- **Step 3.** Request ticket for remote TGS
- **Step 4.** Ticket for remote TGS
- **Step 5.** Request for remote server
- **Step 6.** Ticket for remote server
- **Step 7.** Request remote server

Principal differences between v4 and v5 of Kerberos

- Version 4 (v4) is easier to explain.
- v5 is considered to be standard Kerberos.
- Encryption in v4 implementation of Kerberos uses the Data Encryption Standard (DES), a symmetric-key cipher.
- Encryption in v5 implementation of Kerberos uses the standard CBC (Cipher Block Chaining) mode of DES.
- In v4, inter-realm authentication (cross-realm authentication) among N realms requires on the order of N^2 Kerberos-Kerberos relationships.
- In v5, cross-realm authentication among N realms requires fewer Kerberos-Kerberos relationships.

Thank You